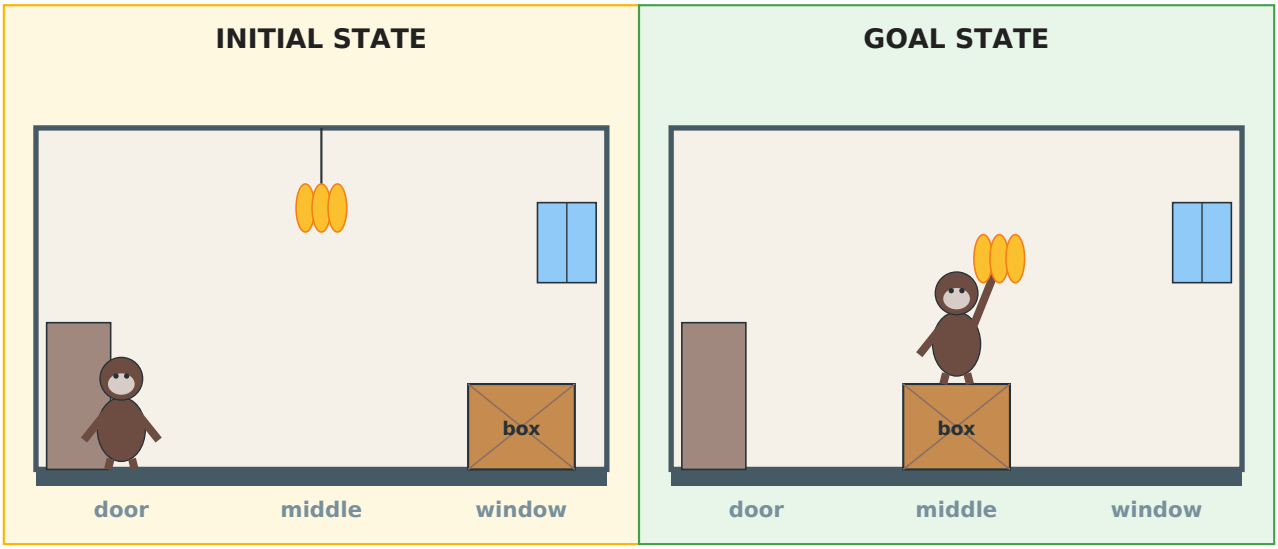
Problem 4: Monkey and Banana

Level: **Basic** · Folder: **04_Monkey_and_Banana/** · Files: domain.pddl, problem.pddl, solution_plan.txt

1. The problem

A hungry monkey is at the **door** of a room. A bunch of bananas hangs from the ceiling in the **middle** of the room, too high to reach from the floor. A box stands near the **window**. The monkey can walk, push the box, climb onto the box and grasp the bananas. Goal: the monkey **has the bananas**.



The goal picture shows one state that satisfies the goal (the final state of the plan). Only the facts in the :goal section are required; anything else may be different.

What you will learn

- Predicates with **zero arguments** (`(on-floor)`, `(has-banana)`).
- One action that changes **two objects at once** (push moves monkey and box together).
- **Restrictions** written as preconditions (no walking while standing on the box).
- A goal that does not mention positions at all. The planner must work out *how* to reach it.

2. Formulating the problem in PDDL

Step 1: Types and objects. One type `location`; objects door, window, middle. The monkey, box and bananas are not objects. Each one has its own predicate.

Step 2: Predicates describe what can be true or false in a state:

Predicate	Meaning
`(monkey-at ?l)`	Monkey is at location ?l.
`(box-at ?l)`	Box is at location ?l.
`(banana-at ?l)`	Bananas hang above location ?l (static).
`(on-floor)`	Monkey stands on the floor.
`(on-box)`	Monkey stands on the box.
`(has-banana)`	Monkey holds the bananas.

Step 3: Actions. Each action has *parameters*, a *precondition* (what must be true to apply it) and an *effect* (facts that become **true (+)** or **false (−)**):

Action	Precondition	Effect
go `?from - location, ?to - location` Walk from ?from to ?to while on the floor.	`(monkey-at ?from)` `(on-floor)`	`+ (monkey-at ?to)` `− (monkey-at ?from)`

push
`?from - location, ?to - location`
Push the box from ?from to ?to. Monkey and box both move.
	`(monkey-at ?from)`
	`(box-at ?from)`
	`(on-floor)`
		`+ (monkey-at ?to)`
		`+ (box-at ?to)`
		`− (monkey-at ?from)`
		`− (box-at ?from)`

climb
`?l - location`
Climb on the box at the monkey's location.
	`(monkey-at ?l)`
	`(box-at ?l)`
	`(on-floor)`
		`+ (on-box)`
		`− (on-floor)`

grasp
`?l - location`
Take the bananas when standing on the box below them.
	`(monkey-at ?l)`
	`(banana-at ?l)`
	`(on-box)`
		`+ (has-banana)`

Step 4: Initial state (`:init`) lists *every* fact that is true at the start. Everything else is false (closed-world assumption). **Step 5: Goal** (`:goal`) lists the facts that must be true at the end.

INITIAL STATE (4 facts)	GOAL

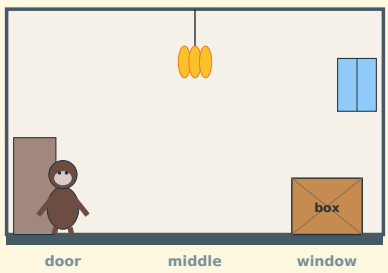
`(monkey-at door)`
`(on-floor)`
`(box-at window)`
`(banana-at middle)`
	`(has-banana)`

Modelling notes

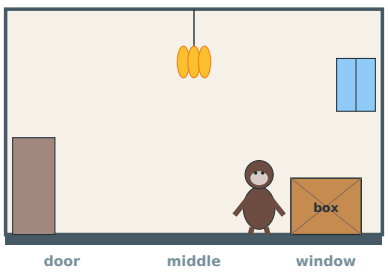
- **on-floor** and **on-box** are complementary. climb adds one and deletes the other.
- **grasp** only adds a fact and deletes nothing. That is allowed.
- The 'insight' (bring the box under the bananas first) is not written anywhere. The planner discovers it by search.

3. Solution plan (4 actions, optimal)

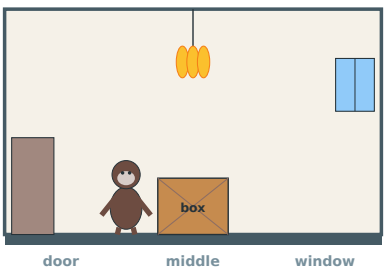
Plan: go to the window, push the box to the middle, climb, grasp (4 steps). The pictures below show the state **after** each action. They were produced by executing the plan on the PDDL model.



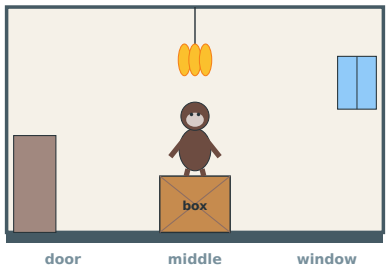
Initial state



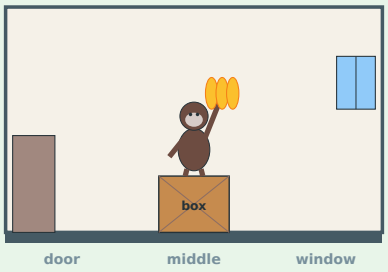
Step 1
`(go door window)`



Step 2
`(push window middle)`



Step 3
`(climb middle)`



Step 4
`(grasp middle)`

4. How to run it

Online (no installation)	Open **editor.planning.domains**, paste *domain.pddl* and *problem.pddl* into two tabs and press **Solve**.
pyperplan (Python)	`pip install pyperplan` `pyperplan domain.pddl problem.pddl` (breadth-first search) `pyperplan -s astar -H lmcut domain.pddl problem.pddl` (optimal A*)
Fast Downward	`fast-downward.py domain.pddl problem.pddl --search "astar(lmcut())"`

5. Lab exercises

1. Add an action **climb-down** and test a problem that starts with the monkey on the box at the window.
2. Start with the box already in the middle. What is the plan now?
3. Add a **stick**: a new predicate (`stick-at ?l`), an action to pick it up, and a second grasp action that works from the floor while holding the stick.

6. PDDL source code (appendix)

domain.pddl

```
;; ============================================================
;; Problem 4 : Monkey and Banana
;; Level     : Basic
;; A monkey must push a box under hanging bananas, climb the
;; box and grab the bananas.
;; ============================================================
(define (domain monkey-banana)
  (:requirements :strips :typing)

  (:types location)

  (:predicates
    (monkey-at ?l - location)   ; monkey is at location ?l
    (box-at ?l - location)      ; box is at location ?l
    (banana-at ?l - location)   ; bananas hang above location ?l
    (on-floor)                  ; monkey stands on the floor
    (on-box)                    ; monkey stands on the box
    (has-banana)                ; monkey holds the bananas
  )

  ;; Walk from one place to another (only while on the floor)
  (:action go
    :parameters (?from - location ?to - location)
    :precondition (and (monkey-at ?from) (on-floor))
    :effect (and (monkey-at ?to)
                 (not (monkey-at ?from)))
  )

  ;; Push the box; monkey and box move together
  (:action push
    :parameters (?from - location ?to - location)
    :precondition (and (monkey-at ?from) (box-at ?from) (on-floor))
    :effect (and (monkey-at ?to) (box-at ?to)
                 (not (monkey-at ?from)) (not (box-at ?from)))
  )

  ;; Climb on the box (monkey must be where the box is)
  (:action climb
    :parameters (?l - location)
    :precondition (and (monkey-at ?l) (box-at ?l) (on-floor))
    :effect (and (on-box)
                 (not (on-floor)))
  )

  ;; Grab the bananas (only reachable from on top of the box)
  (:action grasp
    :parameters (?l - location)
```

```
    :precondition (and (monkey-at ?l) (banana-at ?l) (on-box))
    :effect (has-banana)
  )
)
```

problem.pddl

```
;; Monkey is at the door, box is at the window,
;; bananas hang from the ceiling in the middle of the room.
(define (problem monkey-banana-1)
  (:domain monkey-banana)

  (:objects door window middle - location)

  (:init
    (monkey-at door)
    (on-floor)
    (box-at window)
    (banana-at middle)
  )

  (:goal (has-banana))
)
```

solution_plan.txt

```
1. (go door window)
2. (push window middle)
3. (climb middle)
4. (grasp middle)
```